

# COMP90015: Distributed Systems – Assignment 1

## Multi-threaded Dictionary Server (15 marks)

### Submission Due: 20 April 2026, 5 PM

---

#### Problem Description

Using a client-server architecture, design and implement a multi-threaded server that allows concurrent clients to search the meaning(s) of a word, add a new word, and remove an existing word.

This assignment has been designed to demonstrate the use of two fundamental technologies that have been discussed during the lectures:

- *Sockets*
- *Threads*

Hence, the assignment must make an **EXPLICIT** use of the two above. By explicit, we mean that in your application, sockets and threads must be the lowest level of abstraction for network communication and concurrency.

#### Architecture

- The system will follow a client-server architecture in which multiple clients can connect to a (single) multi-threaded server and perform operations concurrently.
- The multi-threaded server may implement a thread-per-request, thread-per-connection, or worker pool architecture. This is a design decision for you to make.

#### Interaction

- All communication will take place via sockets. These sockets can be either TCP or UDP, however, keep in mind that all communication between clients and server is required to be reliable.
- You are responsible for designing your own message exchange protocol. Some data formats that you may use include XML, JSON, Java Object Serialization, or you may choose to design your own.

#### Failure Model

- All communication between components must be reliable. If you are using TCP, then the reliability guarantees offered by the protocol are sufficient. If you decide to use UDP, then you must implement an infrastructure that offers reliable communication over UDP. For those of you with previous experience using TCP, using UDP may be a rewarding challenge.

- Your server must be able to handle the sudden failovers, and the dictionary must be persisted on the durable storage (e.g. simple file, JSON object, database, etc). This means, if the server process crashes or restarts, the changes made by the clients before the crash/restart should still exist in the dictionary. The server-client connection (in case of TCP) is not required to be re-connected, however, graceful exits and shutdown of client/server is expected. In case of UDP, the dictionary data must be persisted, and appropriate notifications must be delivered to the client in case of server crash/shutdown, in addition to client's graceful exit.
- It is expected that, on both the server and the client side, errors (by means of exception handling) are properly managed. The errors include the following:
  - Input from the console for what concerns the parameters passed as command line.
  - Network communication (server/address not reachable, bad data...).
  - I/O to and from disk (cannot find the dictionary file, error reading the file, etc...).
  - Other errors you might come up with.

The application will be tested and validated against all these errors.

## Functional Requirements

- **Query the meaning(s) of a given word**

The client should implement a function that is used to query the dictionary with the following minimum (additional input/output parameters can be used as required) input and output:

Input: Word to search

Output: Meaning(s) of the word

Error: The client should clearly indicate if the word was not found or if an error occurred. In case of an error, a suitable description of the error should be given to the user.

- **Add a new word**

Add a new word and one or more of its meanings to the dictionary. For the word to be added successfully it should not exist already in the dictionary. Also, attempting to add a word without an associated meaning should result in an error. A new word added by one client should be visible to all other clients of the dictionary server. The minimum input and output parameters are as follows:

Input: Word to add, meaning(s)

Output: Status of the operation (e.g., success, duplicate)

Error: The user should be informed if any errors occurred while performing the operation.

- **Remove an existing word**

Remove a word and all of its associated meanings from the dictionary. A word deleted by one client should not be visible to any of the clients of the dictionary server. This means once a word has been successfully deleted (along with all associated meanings) by any client from the dictionary, this change is immediately reflected in the dictionary. If any client attempts to query the deleted word, it should not be available and must return an appropriate status/error indicating that the word does not exist. It is not required to explicitly notify other clients that a word has been deleted by any client from the dictionary. If the word does not exist in the dictionary, then no action should be taken. The minimum input and output parameters are as follows:

Input: Word to remove

Output: Status of the operation (e.g., success, not found)

Error: The user should be informed if any errors occurred while performing the operation.

- **Adding additional meaning to an existing word**

If multiple meanings already exist, you need to add this new one to the existing list (of course, **if the same meaning already exists, it should NOT be added**). Update made by one client should be visible to any of the clients of the dictionary server. If the word does not exist in the dictionary, then no action should be taken. The minimum input and output parameters are as follows:

Input: Existing Word, new meaning

Output: Status of the operation (e.g., success, not found)

Error: The user should be informed if any errors occurred while performing the operation.

- **Updating existing meaning of an existing word by new one**

If the word and specified meaning already exists, then replace that exiting meaning by new one. Otherwise, no action and flag appropriate message on the client side. Updates made by one client should be visible to any of the clients of the dictionary server. If the word does not exist in the dictionary, then no action should be taken. The minimum input and output parameters are as follows:

Input: Existing Word, existing meaning, new meaning

Output: Status of the operation (e.g., success, not found)

Error: The user should be informed if any errors occurred while performing the operation.

## User Interface

A Graphical User Interface (GUI) is required for this project. A sample client GUI will be provided as part of the assignment description that can be used as-is (with the proper integration with the functional requirements) or you are free to design your GUI. The GUI must be designed in Java and you may use any existing tools/libraries (Java) for this purpose.

A server-GUI or a dashboard is also required to display the # of active connections, operational logs and a manual server start/stop option. The GUI could be as simple as a single frame application with the required functionality. However, you are free to design with any existing tools/libraries (Java) for this purpose.

## Implementation Guidelines

These implementation guidelines must be followed:

- When the server is launched, it loads the dictionary data from a file containing the initial list of words and their meanings. This data is maintained in memory in a structure that enables an efficient word search. When words are added or removed, the data structure is updated to reflect the changes.

A sample command to start the server is:

```
> java -jar DictionaryServer.jar <port> <dictionary-file>
```

Where <port> is the port number where the server will listen for incoming client connections and <dictionary-file> is the path to the file containing the initial dictionary data. **Do not hardcode port number/IP address.**

- When the client is launched, it creates a TCP socket or a UDP socket bound to the server address and port number. This socket remains open for the duration of the client-server interaction. All messages are sent/received through this socket. **Do not hardcode port number/IP address.** Additionally, a configurable sleep or delay argument must be provided to simulate the operation execution time that will be used to verify the thread-safety. The server must be able to handle concurrent threads with simulated delay and only **allow concurrent READS** i.e., at-most 1 client is able to WRITE to the dictionary while multiple clients are able to READ from the dictionary. This implies that concurrent READS and WRITES on the same element/entry/value is prohibited/incompatible and therefore must be handled atomically. Therefore, we may need a shared lock such as ReadWriteLock to prohibit write operations on the shared data in the dictionary.

A sample command to start the client is:

```
> java -jar DictionaryClient.jar <server-address> <server-port>  
<sleep-duration>
```

## Implementation Language

The assignment should be implemented in **Java**. Utilization of technologies such as RMI and JMS are **not allowed**.

## Report

You should write a report describing your system and discussing your design choices. Your report should include:

The problem context in which the assignment has been given

- A brief description of the components of the system. (max. 1-page)
- An overall class design and an interaction diagram. (max. 1-page)
- A critical analysis of the work done, describing the design choices and the reason for making these choices. The following must be answered (max. 2-pages) –
  - Why TCP over UDP is chosen or vice-versa in your design? Explain
  - Why a specific messaging exchange protocol e.g., XML is used over others? Explain
  - How did you ensure or implement the “concurrent READS and restrict concurrent WRITES”? Explain
  - What are the pros/cons of the chosen architecture (client-server), thread-model (thread-per-object or thread-per connection or thread-pool) or thread-safety mechanism (allow concurrent READS and disable concurrent WRITES)?
  - What is the purpose of introducing an intended delay or sleep argument in the server operations? Does this affect the design choices made? Explain how?

Please note that the report is a **WRITTEN** document, do not put only graphs. A report without any descriptive text addressing the problem, architecture, protocols, and the analysis of the work done will not be considered valid.

The length of the report **should not exceed five pages**. A good report is also consistent and contains all the required information for understanding and evaluating the work done. The report must include all the required information listed above. The accepted file format for the report is “PDF” only.

It is important to put your details (name, surname, student id) in:

- The head page of the report. (not counted towards the page limit)
- As a header in each of the files of the software project.

This will help to avoid any mistakes in locating the assignment and its components on both sides.

## Submission

You need to submit **your own work (originally done by you for this subject as per <https://academicintegrity.unimelb.edu.au/>)** the following via LMS:

1. **Your report is in PDF format *only*.**
2. Two *executable jar* files (DictionaryClient.jar and DictionaryServer.jar) that will be used to run your system on the day of the demo.
3. Your source files are in a .ZIP or .TAR archive *only*.

**(These above items should be submitted each one individually – NOT as one zip file containing all. Otherwise, there will be a **penalty of -1 mark**).**

**Submissions will be due on: April 20, 2026; Monday (5 pm).**

### Additional Assessment rule statement:

**For this assignment, No AI use is permitted.**

**This assignment does not allow any use of generative AI tools. This includes use of tools which provide outputs for inclusion in your submission, tools that modify your work, or tools that guide or direct your work.**

### Academic Integrity Expectations and Information

All University of Melbourne students are expected to uphold academic integrity in all aspects of every piece of work that they submit for assessment.

While there is a lot of detail to know, the fundamental issue you must confront is whether you value the University's position on academic integrity and whether you are willing to uphold this value in every piece of work you do. If you align your values to "learning with integrity", implementation of the following is easy and becomes second nature. If you don't value academic integrity, you can still implement the mechanics of the requirements listed under the integrity document, but at some stage you may "forget", or take some short cuts in your assessment, which will probably turn out badly for you either while as a student, or as a professional after graduation.

## Marking

The marking process will be structured by first evaluating whether the assignment (application + report) is compliant with the specification given. This implies the following:

- Use of TCP/UDP sockets and threads for the application.
- proper use of concurrency/thread-safety in the server.
- proper handling of the errors.
- a report with the requested content and format (PDF).
- Server-side features in the GUI.
- Client-side features integrated with GUI.
- No marks for UI design, as long as it is clear and functional

**NOTE 1:** Don't document anything you haven't implemented in the report. This is misconduct and will result in severe penalties.

## Demonstration Schedule and Venue

You are required to provide a demonstration of the working application and will have the opportunity to discuss with the tutors the design and implementation choices made during the demo.

You are free to develop your system where you are more comfortable (at home, on one pc, on your laptop, in the labs...) but keep in mind that the assignment is meant to be a distributed system that works on at least two different machines in order to separate the client from the server.

**We will announce the demo date, time, and venue closer to the due date.** Each tutor will hold 2-3 demo sessions and you will be required to showcase your system in one of the sessions held by the tutor of the workshop in which you are enrolled.

If you need any clarification on the assignment, kindly ask questions during the tutorials or in the LMS forum, so that all students benefit from it.

**NOTE 2:** Demo is going to be campus-based, your Tutor will announce the demo schedule. **DEMO is mandatory and if you DO NOT demonstrate, there will be penalties of -5 Marks.** For the demo, you need to bring your own computer/laptop.

## Penalties for late submissions of assignments

Assignments submitted late will be penalized in the following way:

- 1 day late: -1 mark
- 2 days late: -2 marks
- 3 days late: -3 marks
- 4 days late: -4 marks
- etc.